Exercise Sheet 12

Retire MACRO-9000 · two sessions

House rules for the project.

- First: `pip install -r requirements.txt`. Then create **two** files in `my_work/`: `paygap.py` (your functions; it never prints and never reads a file by itself) and, per exercise, a small `checkN.py` that imports it and prints. In session 2 you add `report.py`.
- Run everything from the lesson folder: `python my_work/check1.py`. Because the check script lives next to `paygap.py`, `import paygap` just works.
- Every function gets a one-line docstring. Future Jen will thank present Jen.
- Hand in at the end of **each** session: commit, push to `lesson-12`. Open the pull request after session 2.

The rules of the game: UK gender pay gap reporting

read first

Employers with 250+ staff must publish six figures every year. Brightwell has 200, so it's voluntary, which is exactly why Gerald wants it done properly. All gaps are a percentage of **men's** pay:

```
gap % = (men - women) / men * 100          # positive: men earn more
```

FIGURE	BASED ON
1. Mean hourly pay gap	mean hourly rate of men vs women
2. Median hourly pay gap	median hourly rate of men vs women
3. Mean bonus gap	mean bonus, counting only people who got one
4. Median bonus gap	median bonus, same people
5. Proportion receiving a bonus	% of all men and % of all women who got any bonus
6. Pay quartiles	sort everyone by hourly rate, cut into four equal bands, % women in each

Hourly rate = `annual_salary / 52 / weekly_hours`. That's the whole reason part-timers don't skew the result.

Session 1 · the numbers

Exercise 1 Load and add the hourly rate

15 min · `paygap.py`, `check1.py`

> **From Priya (Payroll):** *"Start with the only number that matters: what does each person earn per hour? Tara in Warehouse does 18.75 hours; her salary looks tiny and her hourly rate is fine. MACRO-9000 never understood that."*

GIVEN (TOP OF PAYGAP.PY)

```
"""paygap.py: the Brightwell gender pay gap, by the book."""
import pandas as pd
```

(a) Write `load_payroll(path)`: read the CSV, add a column `hourly_rate`, return the DataFrame.

(b) In `check1.py`: `import paygap`, load `data/brightwell_payroll.csv`, print `df.shape`, the first three rows of `employee_id, gender, weekly_hours, annual_salary, hourly_rate` rounded to 2 decimals, and the mean hourly rate of women and of men.

EXPECTED OUTPUT

```
(200, 13)
  employee_id gender  weekly_hours  annual_salary  hourly_rate
0     EMP1092      M          37.5          41900        21.49
1     EMP1178      F          37.5          26550        13.62
2     EMP1105      F          37.5          24950        12.79
Mean hourly rate, women: £14.95
Mean hourly rate, men:   £18.62
```

Exercise 2 The hourly pay gap

15 min · `check2.py`

> **From Gerald (MD):** *"Two numbers. Mean and median. I've been told they're different and that the difference 'tells a story'. I would like the story."*

(a) In `paygap.py`, write `gap_percent(men, women)`: the formula from the rules, rounded to 1 decimal, returned as a plain `float` (wrap it: `float(round(..., 1))`).

(b) Write `hourly_gap(df)`: pick men's and women's hourly rates and return a dict with keys `mean_gap` and `median_gap`, both made with `gap_percent`.

(c) `check2.py` prints `gap_percent(20.00, 17.40)`, `gap_percent(18.50, 18.90)` and `hourly_gap(df)`.

EXPECTED OUTPUT

```
13.0
-2.2
{'mean_gap': 19.7, 'median_gap': 18.7}
```

Hint: Why both? The mean is pulled up by a few very high salaries (mostly men). The median is the "typical" person. If the mean gap is much bigger than the median gap, the problem is at the top.

Exercise 3 The bonus gap

15 min · `check3.py`

> **From Siobhan (Reward):** *"Bonuses are my department and I already know I won't like this. Only count people who actually got one. A zero isn't a small bonus, it's no bonus."*

(a) Write `bonus_gap(df)`. Keep only rows with `bonus > 0` for the gaps. Return a dict with `mean_bonus_gap`, `median_bonus_gap`, `pct_men_with_bonus` and `pct_women_with_bonus` (the last two out of *all* men / *all* women, 1 decimal).

(b) `check3.py` loops over the dict with `.items()` and prints `key: value` per line.

EXPECTED OUTPUT

```
mean_bonus_gap: 22.8
median_bonus_gap: 30.8
pct_men_with_bonus: 66.7
pct_women_with_bonus: 45.8
```

Exercise 4 Pay quartiles

15 min · `check4.py`

> **From Priya:** *"The quartiles are the picture the board will remember. Who's in the bottom 25%? Who's in the top?"*

(a) Write `pay_quartiles(df)`: `pd.qcut(df["hourly_rate"], 4, labels=[...])` with the labels `Lower, Lower middle, Upper middle, Upper`; then `pd.crosstab(bands, df["gender"], normalize="index") * 100`, rounded to 1 decimal.

(b) Rename the columns to `pct_women` and `pct_men`, name the index `quartile` (`table.index.name = "quartile"`), and return the columns in that order.

(c) `check4.py` prints it.

EXPECTED OUTPUT

```
gender         pct_women  pct_men
quartile
Lower               72.0     28.0
Lower middle        46.0     54.0
Upper middle        30.0     70.0
Upper               18.0     82.0
```

Hint: Read it like a sentence: "72% of the lowest-paid quarter are women; 18% of the highest-paid quarter are women." Then look at the headcount: 41.5% of Brightwell is women.

End of session 1. Commit (`git add . && git commit -m "Pay gap: session 1, the six figures"`) and push. Tell Lars the one number that surprised you most.

Session 2 · the report

Exercise 5 By department

15 min · `check5.py`

> **From Mo (IT):** *"One number for the whole company hides everything. Where is the gap actually coming from? (If it's IT, I'd like to know before Gerald does.)"*

(a) Write `by_department(df)`: loop over `df.groupby("department")`; for each group collect `employees`, `pct_women` (1 decimal) and `median_gap` (with `gap_percent` on the medians) into a list of dicts.

(b) Turn the list into a DataFrame, `set_index("department")`, sort by `median_gap` descending, return it.

(c) `check5.py` prints it.

EXPECTED OUTPUT

```
            employees  pct_women  median_gap
department
HR                 12       66.7        48.6
IT                 20       35.0        37.5
Warehouse          25       24.0        36.0
Sales              35       40.0        31.1
Marketing          15       73.3        21.0
Production         60       33.3        11.8
Finance            18       50.0        11.0
Design             15       53.3         2.6
```

Hint: HR is 67% women and has the biggest gap. That's not a contradiction; it's the whole point of the quartiles. Who are the four men in HR?

Exercise 6 The console report

20 min · `report_console.py`

> **From Gerald:** *"One sentence I can say out loud at the board without looking at a slide. Then the detail underneath for the people who like detail."*

(a) In `paygap.py`, write `one_sentence(df)`: pence per pound is `round(100 - median_gap)`. Return exactly the sentence shown in the output below.

(b) Write `report_console.py`: load the data, call your five functions once each, and print the report below. The header line is `"=" * 48`; the percentages are right-aligned in 5 characters (`{value:>5}`).

EXPECTED OUTPUT

```
BRIGHTWELL UMBRELLAS LTD: GENDER PAY GAP REPORT
================================================
Employees: 200 (83 women, 117 men)
Mean hourly pay gap:     19.7%
Median hourly pay gap:   18.7%
Mean bonus gap:          22.8%
Median bonus gap:        30.8%
Received a bonus:       women 45.8%, men 66.7%

gender        pct_women  pct_men
quartile
Lower              72.0     28.0
Lower middle       46.0     54.0
Upper middle       30.0     70.0
Upper              18.0     82.0

            employees  pct_women  median_gap
department
HR                 12       66.7        48.6
IT                 20       35.0        37.5
Warehouse          25       24.0        36.0
Sales              35       40.0        31.1
Marketing          15       73.3        21.0
Production         60       33.3        11.8
Finance            18       50.0        11.0
Design             15       53.3         2.6

At Brightwell, women earn 81p for every £1 that men earn (median hourly pay gap: 18.7%,
mean: 19.7%).
```

Exercise 7 The chart and the workbook

25 min · `report.py`

> **From Priya:** *"And now the version for grown-ups: a picture of the quartiles and an Excel file with the six figures on the first tab. Then we print it, put it on Gerald's desk, and unplug MACRO-9000."*

(a) Copy `report_console.py` to `report.py`. Add the matplotlib/openpyxl imports from lesson 11 and `OUT = Path("my_work/output")`, created with `mkdir`.

(b) Chart: a **stacked** bar per quartile. Women first (`plt.bar(quartiles.index, quartiles["pct_women"], label="Women")`), then men on top with `bottom=quartiles["pct_women"]`. Title, y-label, legend, save as `OUT / "quartiles.png"`.

(c) Workbook `OUT / "pay_gap_report.xlsx"` with three sheets: **Summary** (a two-column DataFrame `measure` / `value` holding the six figures, `index=False`), **Quartiles**, **By department**.

(d) Re-open it with openpyxl and, `for ws in wb.worksheets:`, make row 1 bold white on navy, set column A to width 28, freeze `A2`. Save. Print the last line below.

EXPECTED OUTPUT

Everything from Exercise 6, then:

```
Saved my_work/output/pay_gap_report.xlsx and my_work/output/quartiles.png
```

Hint: Open the workbook and the PNG. That's the report. Print the PNG, walk it over to Gerald's desk, and watch his face.

Bug hunt Dave's Leftovers #12

15 min · `bughunt/macro9000_bug.py`

The last leftover. In 2019 Dave ported MACRO-9000's pay gap calculation to Python. It told the board the gap was **24.6%** for three years running. Your report says 19.7%. The board wants to know who's right. **Three** bugs: one crashes, one divides by the wrong thing, one calls something a median that isn't.

EXPECTED OUTPUT ONCE FIXED

```
MACRO-9000 PAY GAP CHECK
Mean hourly gap:   19.7%
Median hourly gap: 18.7%
MACRO-9000 agrees with Jen. MACRO-9000 may now retire.
```

One commit per bug. Then open the pull request, title: **"Retire MACRO-9000"**. In the description, paste Gerald's sentence and the one thing you'd tell the board that the numbers don't say.